

Snowflake + dbt: FinTrack Analytics

Construisez un entrepôt de données structuré et fiable pour alimenter des tableaux de bord financiers à partir de données brutes.

DURÉE ESTIMÉE

3 à 5 jours

NIVEAU

Débutant

PRÉREQUIS

Bases en SQL, notions de modélisation dimensionnelle, compte Snowflake (trial gratuit 30 jours)

Scénario

Vous venez d'intégrer **FinTrack Analytics**, une jeune fintech qui propose une application de gestion de finances personnelles. L'entreprise collecte des données sur les comptes bancaires de ses utilisateurs, leurs transactions quotidiennes, les catégories de dépenses et les budgets définis par chaque client.

Jusqu'à présent, les données brutes sont stockées en vrac dans un schéma Snowflake sans aucune transformation. Le directeur data vous confie la mission suivante :

« Nous avons besoin d'un entrepôt de données structuré et fiable pour alimenter nos tableaux de bord financiers. Mettez en place une pipeline dbt qui transforme nos données brutes en modèles analytiques exploitables. »

Objectifs pédagogiques

À l'issue de ce projet, vous serez capable de :

1. Configurer un environnement Snowflake (base, schémas, rôles, entrepôt)
2. Charger des données brutes dans Snowflake via `COPY INTO`
3. Initialiser et structurer un projet dbt connecté à Snowflake
4. Créer des modèles staging, intermédiaires et marts
5. Implémenter des tests de qualité de données (generic et singular)
6. Documenter vos modèles avec des fichiers YAML
7. Produire un lineage exploitable via `dbt docs`

1

Configuration de l'environnement Snowflake

1.1 — Créer les ressources Snowflake

Connectez-vous à votre compte Snowflake et créez les ressources nécessaires au projet :

- Une base de données dédiée au projet
- Trois schémas : données brutes, transformations dbt, modèles analytiques finaux
- Un entrepôt de calcul de taille XS
- Un rôle et un utilisateur dédiés au projet

💡 **Indice :** La convention de nommage recommandée est `FINTRACK_DB` pour la base, et `RAW`, `STAGING`, `MARTS` pour les schémas.

```
-- Exemple de structure attendue (à compléter)CREATE DATABASE ...;
CREATE SCHEMA ...;
CREATE WAREHOUSE ... WITH WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60 AUTO_RESUME = TRUE;

-- Rôle et permissionsCREATE ROLE ...;
GRANT USAGE ON DATABASE ... TO ROLE ...;
GRANT USAGE ON WAREHOUSE ... TO ROLE ...;
-- Pensez à accorder les droits sur chaque schéma
```

1.2 — Modèle de données brutes

Voici le schéma des quatre tables sources à créer dans le schéma `RAW` :

`raw_comptes`

Colonne	Type	Description
<code>id</code>	INTEGER	Identifiant unique du compte
<code>nom_client</code>	VARCHAR(100)	Nom complet du client
<code>email</code>	VARCHAR(150)	Adresse email du client
<code>type_compte</code>	VARCHAR(20)	Type : courant, epargne, joint
<code>date_ouverture</code>	DATE	Date d'ouverture du compte
<code>solde_initial</code>	DECIMAL(12,2)	Solde à l'ouverture du compte
<code>statut</code>	VARCHAR(10)	Statut : actif, inactif, cloture

raw_transactions

Colonne	Type	Description
<code>id</code>	INTEGER	Identifiant unique de la transaction
<code>compte_id</code>	INTEGER	Référence vers raw_comptes.id
<code>date_transaction</code>	TIMESTAMP	Date et heure de la transaction
<code>montant</code>	DECIMAL(10,2)	Montant (toujours positif)
<code>type_operation</code>	VARCHAR(10)	Type : debit ou credit
<code>categorie_id</code>	INTEGER	Référence vers raw_categories.id
<code>description</code>	VARCHAR(255)	Libellé de la transaction
<code>statut</code>	VARCHAR(15)	Statut : validee, en_attente, rejetee

raw_categories

Colonne	Type	Description
<code>id</code>	INTEGER	Identifiant unique de la catégorie
<code>nom</code>	VARCHAR(50)	Nom de la catégorie
<code>type</code>	VARCHAR(10)	Type : depense ou revenu
<code>groupe</code>	VARCHAR(30)	Groupe parent (Quotidien, Logement, Loisirs...)

raw_budgets

Colonne	Type	Description
<code>id</code>	INTEGER	Identifiant unique du budget
<code>compte_id</code>	INTEGER	Référence vers raw_comptes.id
<code>categorie_id</code>	INTEGER	Référence vers raw_categories.id
<code>mois</code>	DATE	Premier jour du mois concerné
<code>montant_prevu</code>	DECIMAL(10,2)	Montant budgété pour ce mois

1.3 – Charger les données d'exemple

Créez les tables ci-dessus dans le schéma `RAW`, puis insérez les données d'exemple suivantes.

```
-- CATÉGORIES
INSERT INTO raw.raw_categories (id, nom, type, groupe) VALUES
(1, 'Salaire', 'revenu', 'Revenus'),
(2, 'Freelance', 'revenu', 'Revenus'),
(3, 'Alimentation', 'depense', 'Quotidien'),
(4, 'Transport', 'depense', 'Quotidien'),
(5, 'Loyer', 'depense', 'Logement'),
(6, 'Électricité', 'depense', 'Logement'),
(7, 'Restaurant', 'depense', 'Loisirs'),
(8, 'Abonnements', 'depense', 'Loisirs'),
(9, 'Épargne versée', 'depense', 'Épargne'),
(10, 'Remboursement', 'revenu', 'Divers'),
(11, 'Santé', 'depense', 'Quotidien'),
(12, 'Vêtements', 'depense', 'Loisirs');
```

```
-- COMPTESINSERT INTO raw.raw_comptes
(id, nom_client, email, type_compte, date_ouverture, solde_initial, statut)
VALUES
(1, 'Alice Dupont', 'alice.dupont@email.fr', 'courant', '2023-01-15', 2500.00,
'actif'),
(2, 'Bob Martin', 'bob.martin@email.fr', 'courant', '2023-03-22', 1800.00,
'actif'),
(3, 'Claire Leroy', 'claire.leroy@email.fr', 'epargne', '2023-06-01', 5000.00,
'actif'),
(4, 'David Moreau', 'david.moreau@email.fr', 'joint', '2022-11-10', 3200.00,
'actif'),
(5, 'Emma Bernard', 'emma.bernard@email.fr', 'courant', '2024-01-05', 900.00,
'inactif'),
(6, 'Frank Petit', 'frank.petit@email.fr', 'courant', '2023-09-18', 1500.00,
'cloture');
```

```
-- TRANSACTIONS (échantillon - janvier à mars 2024)INSERT INTO raw.raw_transactions
(id, compte_id, date_transaction, montant, type_operation,
categorie_id, description, statut) VALUES-- Alice Dupont - Janvier 2024
(1, 1, '2024-01-02 09:00:00', 2800.00, 'credit', 1, 'Salaire janvier',
'validee'),
(2, 1, '2024-01-03 14:30:00', 850.00, 'debit', 5, 'Loyer janvier',
'validee'),
(3, 1, '2024-01-05 12:15:00', 65.40, 'debit', 3, 'Courses Carrefour',
'validee'),
(4, 1, '2024-01-08 19:45:00', 42.00, 'debit', 7, 'Restaurant Le Bistrot',
'validee'),
(5, 1, '2024-01-10 08:00:00', 75.00, 'debit', 4, 'Abonnement Navigo',
'validee'),
(6, 1, '2024-01-15 10:00:00', 15.99, 'debit', 8, 'Netflix',
'validee'),
(7, 1, '2024-01-20 16:00:00', 200.00, 'debit', 9, 'Virement épargne',
'validee'),
(8, 1, '2024-01-22 11:30:00', 34.50, 'debit', 11, 'Pharmacie',
'validee'),
-- Alice Dupont - Février 2024
(9, 1, '2024-02-01 09:00:00', 2800.00, 'credit', 1, 'Salaire février',
'validee'),
(10, 1, '2024-02-03 14:00:00', 850.00, 'debit', 5, 'Loyer février',
'validee'),
(11, 1, '2024-02-06 13:00:00', 78.20, 'debit', 3, 'Courses Monoprix',
'validee'),
(12, 1, '2024-02-10 08:00:00', 75.00, 'debit', 4, 'Abonnement Navigo',
'validee'),
```

```
(13, 1, '2024-02-14 20:00:00', 89.00, 'debit', 7, 'Restaurant Saint-Valentin',
'validee'),
(14, 1, '2024-02-15 10:00:00', 15.99, 'debit', 8, 'Netflix',
'validee'),
(15, 1, '2024-02-20 16:00:00', 200.00, 'debit', 9, 'Virement épargne',
'validee'),
-- Alice Dupont - Mars 2024
(16, 1, '2024-03-01 09:00:00', 2800.00, 'credit', 1, 'Salaire mars',
'validee'),
(17, 1, '2024-03-03 14:00:00', 850.00, 'debit', 5, 'Loyer mars',
'validee'),
(18, 1, '2024-03-07 10:30:00', 92.10, 'debit', 3, 'Courses Leclerc',
'validee'),
(19, 1, '2024-03-12 19:00:00', 55.00, 'debit', 7, 'Restaurant Sushi Shop',
'en_attente'),
(20, 1, '2024-03-15 10:00:00', 15.99, 'debit', 8, 'Netflix',
'validee'),
-- Bob Martin - Janvier 2024
(21, 2, '2024-01-03 09:00:00', 2200.00, 'credit', 1, 'Salaire janvier',
'validee'),
(22, 2, '2024-01-05 15:00:00', 650.00, 'debit', 5, 'Loyer janvier',
'validee'),
(23, 2, '2024-01-07 18:00:00', 35.00, 'debit', 7, 'Kebab Chez Ali',
'validee'),
(24, 2, '2024-01-09 09:30:00', 45.80, 'debit', 3, 'Courses Lidl',
'validee'),
(25, 2, '2024-01-12 14:00:00', 350.00, 'credit', 2, 'Mission freelance design',
'validee'),
(26, 2, '2024-01-18 11:00:00', 9.99, 'debit', 8, 'Spotify',
'validee'),
(27, 2, '2024-01-25 08:00:00', 120.00, 'debit', 12, 'Zara soldes',
'validee'),
-- Bob Martin - Février 2024
(28, 2, '2024-02-02 09:00:00', 2200.00, 'credit', 1, 'Salaire février',
'validee'),
(29, 2, '2024-02-04 15:00:00', 650.00, 'debit', 5, 'Loyer février',
'validee'),
(30, 2, '2024-02-08 13:00:00', 52.30, 'debit', 3, 'Courses Auchan',
'validee'),
(31, 2, '2024-02-12 20:00:00', 28.00, 'debit', 7, 'Pizza Hut',
'rejetee'),
(32, 2, '2024-02-15 10:00:00', 9.99, 'debit', 8, 'Spotify',
'validee'),
(33, 2, '2024-02-20 16:00:00', 500.00, 'credit', 2, 'Mission freelance dev',
'validee'),
-- David Moreau (compte joint) - Janvier 2024
(34, 4, '2024-01-02 09:00:00', 4500.00, 'credit', 1, 'Salaire David',
```

```
'validee'),
(35, 4, '2024-01-02 09:30:00', 2100.00, 'credit', 1, 'Salaire conjointe',
'validee'),
(36, 4, '2024-01-05 14:00:00', 1200.00, 'debit', 5, 'Loyer janvier',
'validee'),
(37, 4, '2024-01-06 17:00:00', 85.00, 'debit', 6, 'EDF facture',
'validee'),
(38, 4, '2024-01-10 12:00:00', 156.30, 'debit', 3, 'Courses familiales',
'validee'),
(39, 4, '2024-01-15 20:00:00', 110.00, 'debit', 7, 'Restaurant en famille',
'validee'),
(40, 4, '2024-01-20 10:00:00', 500.00, 'debit', 9, 'Épargne mensuelle',
'validee'),
-- Claire Leroy (épargne) – Janvier 2024
(41, 3, '2024-01-15 10:00:00', 300.00, 'credit', 9, 'Virement depuis courant',
'validee'),
(42, 3, '2024-02-15 10:00:00', 300.00, 'credit', 9, 'Virement depuis courant',
'validee'),
(43, 3, '2024-03-15 10:00:00', 300.00, 'credit', 9, 'Virement depuis courant',
'validee'),
-- Emma Bernard (inactive)
(44, 5, '2024-01-10 09:00:00', 1500.00, 'credit', 1, 'Salaire janvier',
'validee'),
(45, 5, '2024-01-12 14:00:00', 45.00, 'debit', 3, 'Courses',
'validee');
```



```

-- BUDGETS (Janvier à Mars 2024 – Alice et Bob)INSERT INTO raw.raw_budgets
(id, compte_id, categorie_id, mois, montant_prevu) VALUES-- Alice – Janvier
(1, 1, 3, '2024-01-01', 150.00), (2, 1, 4, '2024-01-01', 80.00),
(3, 1, 5, '2024-01-01', 850.00), (4, 1, 7, '2024-01-01', 60.00),
(5, 1, 8, '2024-01-01', 20.00), (6, 1, 9, '2024-01-01', 200.00),
-- Alice – Février
(7, 1, 3, '2024-02-01', 150.00), (8, 1, 4, '2024-02-01', 80.00),
(9, 1, 5, '2024-02-01', 850.00), (10, 1, 7, '2024-02-01', 60.00),
(11, 1, 8, '2024-02-01', 20.00), (12, 1, 9, '2024-02-01', 200.00),
-- Alice – Mars
(13, 1, 3, '2024-03-01', 160.00), (14, 1, 5, '2024-03-01', 850.00),
(15, 1, 7, '2024-03-01', 50.00), (16, 1, 8, '2024-03-01', 20.00),
-- Bob – Janvier
(17, 2, 3, '2024-01-01', 100.00), (18, 2, 5, '2024-01-01', 650.00),
(19, 2, 7, '2024-01-01', 50.00), (20, 2, 8, '2024-01-01', 15.00),
(21, 2, 12, '2024-01-01', 80.00),
-- Bob – Février
(22, 2, 3, '2024-02-01', 100.00), (23, 2, 5, '2024-02-01', 650.00),
(24, 2, 7, '2024-02-01', 50.00), (25, 2, 8, '2024-02-01', 15.00);

```

2 Initialisation du projet dbt

2.1 – Installer dbt et créer le projet

Installez dbt avec l'adaptateur Snowflake, puis initialisez un nouveau projet.

```
$ pip install dbt-snowflake
$ dbt init fintrack_analytics
```

Configurez le fichier `~/.dbt/profiles.yml` pour connecter dbt à votre environnement Snowflake :

```
fintrack_analytics:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: '<votre_compte>'      # ex : xy12345.eu-west-1
      user: '<votre_utilisateur>'
      password: '<votre_mot_de_passe>'
      role: '...'                  # le rôle créé en Partie 1
      database: '...'
      warehouse: '...'
      schema: '...'               # schéma par défaut
      threads: 2
```

Vérifiez la connexion :

```
$ dbt debug
```

2.2 – Structurer le projet

Organisez votre projet dbt selon la structure suivante :

```

fintrack_analytics/
├─ models/
│   ├─ staging/           ← Nettoyage des données brutes
│   │   ├─ _stg_sources.yml
│   │   ├─ stg_comptes.sql
│   │   ├─ stg_transactions.sql
│   │   ├─ stg_categories.sql
│   │   └─ stg_budgets.sql
│   ├─ intermediate/     ← Jointures et enrichissements
│   │   └─ int_transactions_enrichies.sql
│   └─ marts/            ← Modèles analytiques finaux
│       ├─ dim_comptes.sql
│       ├─ dim_categories.sql
│       ├─ fct_transactions.sql
│       ├─ mart_solde_mensuel.sql
│       └─ mart_budget_vs_reel.sql
├─ tests/
│   └─ assert_solde_mensuel_positif.sql
├─ dbt_project.yml
└─ packages.yml

```

2.3 – Configurer le dbt_project.yml

Configurez les matérialisations par défaut pour chaque couche :

staging

view

Données changeantes

intermediate

ephemeral

CTE injectée

marts

table

Modèles finaux BI

Les modèles staging sont la première couche de transformation. Leur rôle :

- Déclarer les sources (fichier `_stg_sources.yml`)
- Renommer les colonnes pour suivre une convention cohérente
- Caster les types de données
- Filtrer les données invalides éventuelles

3.1 – Déclarer les sources

Créez le fichier `_stg_sources.yml` qui déclare vos quatre tables brutes comme sources dbt.

 **Indice :** Utilisez la fonction `source()` dans vos modèles pour référencer ces tables. Le YAML doit contenir le `database` , le `schema` , et la liste des `tables` .

3.2 – Créer les modèles staging

Pour chaque table source, créez un modèle staging avec les transformations suivantes :

`stg_comptes.sql`

- Sélectionner toutes les colonnes de `raw_comptes`
- Renommer `id` en `compte_id`
- Convertir `statut` en minuscules
- Référencer la source avec

stg_transactions.sql

- Sélectionner toutes les colonnes de `raw_transactions`
- Renommer `id` en `transaction_id`
- Caster `date_transaction` en `TIMESTAMP_NTZ`
- Ajouter `montant_signe` : +montant si crédit, -montant si débit

💡 Utilisez une expression `CASE WHEN` pour calculer `montant_signe`.

stg_categories.sql

- Renommer `id` → `categorie_id`
- Renommer `nom` → `nom_categorie`
- Renommer `type` → `type_categorie` (éviter le mot réservé SQL)

stg_budgets.sql

- Renommer `id` → `budget_id`
- S'assurer que `mois` est de type `DATE`

`int_transactions_enrichies.sql`

Ce modèle enrichit les transactions en joignant comptes et catégories. Il doit contenir :

- Toutes les colonnes de `stg_transactions`
- `nom_client` et `type_compte` depuis `stg_comptes`
- `nom_categorie`, `type_categorie` et `groupe` depuis `stg_categories`
- Un champ `mois_transaction` (premier jour du mois)

💡 Utilisez `DATE_TRUNC('month', date_transaction)` pour extraire le premier jour du mois.

Rappel : Ce modèle est matérialisé en `ephemeral` — il ne crée pas de table. Il sera injecté comme CTE via `WITH`.

5.1 – dim_comptes.sql

Table de dimension des comptes clients. Ajoutez un champ `anciennete_jours` calculé comme la différence entre aujourd'hui et `date_ouverture`.

💡 `DATEDIFF('day', date_ouverture, CURRENT_DATE())`

5.2 – dim_categories.sql

Table de dimension des catégories. Reprenez simplement les colonnes de `stg_categories`.

5.3 – fct_transactions.sql

Table de faits des transactions. À partir de `int_transactions_enrichies`, sélectionnez :

<code>transaction_id</code>	<code>compte_id</code>	<code>categorie_id</code>	<code>date_transaction</code>	
<code>mois_transaction</code>	<code>montant</code>	<code>montant_signe</code>	<code>type_operation</code>	<code>nom_client</code>
<code>nom_categorie</code>	<code>groupe</code>	<code>statut</code>		

Filtrez pour ne garder que les transactions avec `statut = 'validee'`.

5.4 – mart_solde_mensuel.sql

Calcule le solde mensuel de chaque compte. Pour chaque couple (compte, mois) :

- Total des crédits du mois
- Total des débits du mois
- Solde net du mois (crédits – débits)
- Solde cumulé depuis l'ouverture du compte

💡 Le solde cumulé combine le `solde_initial` avec la somme cumulée des `montant_signe`, via une window function `SUM(...) OVER (PARTITION BY ... ORDER BY ...)`.

5.5 – mart_budget_vs_reel.sql

Compare les dépenses réelles aux budgets prévus par (compte, catégorie, mois) :

compte_id	nom_client	categorie_id	nom_categorie	mois	montant_prevu
montant_reel	ecart	depassement			

💡 Utilisez un `FULL OUTER JOIN` entre budgets agrégés et dépenses agrégées.

6 Tests et qualité de données

6.1 – Tests génériques (YAML)

Ajoutez des tests dans vos fichiers YAML de documentation. Au minimum :

Modèle	Colonne	Tests
stg_comptes	compte_id	unique, not_null
stg_comptes	type_compte	accepted_values : courant, epargne, joint
stg_comptes	statut	accepted_values : actif, inactif, cloture
stg_transactions	transaction_id	unique, not_null
stg_transactions	compte_id	relationships → stg_comptes
stg_categories	categorie_id	unique, not_null
fct_transactions	transaction_id	unique, not_null

 **Indice :** Le test `relationships` vérifie l'intégrité référentielle :

```
- relationships:
  to: ref('stg_comptes')
  field: compte_id
```

6.2 – Test singulier (SQL)

Créez un test personnalisé `tests/assert_solde_mensuel_positif.sql` qui vérifie qu'aucun compte `epargne` n'a un solde cumulé négatif.

Rappel : Un test singulier dbt est un fichier SQL qui retourne les lignes en erreur. Zéro lignes = test réussi.

6.3 — Exécuter les tests

```
$ dbt test
```

Analysez les résultats. Si un test échoue, identifiez la cause et corrigez votre modèle ou vos données.

7.1 — Documenter vos modèles

Créez des fichiers YAML de documentation pour chaque couche. Ajoutez au minimum :

- Une **description** pour chaque modèle
- Une **description** pour les colonnes clés de chaque modèle mart

7.2 — Générer et consulter la documentation

```
$ dbt docs generate  
$ dbt docs serve
```

Naviguez dans l'interface et vérifiez :

- Le **lineage** est cohérent : sources → staging → intermediate → marts
- Les descriptions apparaissent sur chaque modèle
- Les tests sont visibles dans la documentation



Aller plus loin — bonus

8.1 — Ajouter dbt-utils

Installez le package `dbt-utils` et utilisez `generate_surrogate_key` pour créer des clés de substitution.

```
# packages.yml
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.0.0", "<2.0.0"]
```

```
$ dbt deps
```

8.2 — mart_top_depenses.sql

Classez les catégories de dépenses par montant total pour chaque client et ne retenez que le top 3.

```
💡 ROW_NUMBER() OVER (PARTITION BY compte_id ORDER BY total DESC) + filtrez avec WHERE rang  
≤ 3 .
```

8.3 — Snapshots (suivi historique)

Implémentez un snapshot sur `raw_comptes` pour suivre les changements de statut (stratégie `check` sur la colonne `statut`).

Livrables attendus

1

Le script SQL Snowflake

Création de l'environnement (base, schémas, rôle, entrepôt, tables, données)

2

Le projet dbt complet

Modèles staging, intermédiaire et marts fonctionnels · Tests génériques et singulier · Documentation YAML · dbt_project.yml configuré

3

Capture d'écran du lineage

dbt docs montrant la chaîne complète sources → staging → intermediate → marts

4

Un court rapport (10-15 lignes)

Répondez aux questions : Différence vue/table/éphémère dans dbt ? Pourquoi séparer staging, intermediate et marts ? Quel avantage du test relationships ?

Ressources utiles

- [Documentation officielle dbt](#)
- [Documentation Snowflake](#)
- [dbt Learn — Cours gratuit](#)
- [dbt-utils sur le Hub](#)
- [Guide des matérialisations dbt](#)